Basic (critique your own Stage-1 design) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `design_self_review.md`: a weak design decision caught on paper costs you essentially nothing; the exact same decision caught three weeks into building costs you three real weeks. This exercise is that deliberate pause, done on purpose, before Stage 2 ever starts. **If not:** unneeded pieces would get built anyway, and missing ones would only be discovered when a real requirement fails in production.

Write your own version first, in `practice/design_self_review.md`, then compare.

Since this document's real requirements are handed to you live, this solution walks through the technique on a realistic stand-in example: a capstone with requirements "handle 50,000 requests/day, p95 latency under 2 seconds, cost under $200/month, must never expose raw customer data to the LLM." (In this example, the capstone design added a fact-checker agent to Project 4's team, to check each draft's claims against the RAG sources.)

Solution

Story: this checks both directions deliberately — every component on the page needs a requirement justifying it, and every requirement needs a component that actually satisfies it. The second direction is what finds what's genuinely *missing*, which is far easier to overlook than something extra and unneeded. **If not:** a stated cost limit would have no actual component enforcing it anywhere, and nothing on the page would visibly look wrong.

Run the full two-direction traceability check.

Direction 1 — every component traces to a requirement:

Component	Requirement it satisfies	Verdict
Supervisor pattern (Doc11) with dynamic routing	Functional: fact-check failures must route back to the writer, a fixed pipeline can't do this	Keep — justified, not just "the standard pattern"
Redis cache in front of the retriever	Non-functional: p95 latency under 2s at 50K req/day; re-embedding repeated queries is the dominant cost	Keep

Component	Requirement it satisfies	Verdict
Second, parallel fact-checker agent	None found	Cut. If a later reliability audit shows single-checker accuracy is insufficient, this becomes justified — right now it's speculative
Field-level redaction layer before any LLM call	Security: "must never expose raw customer data to the LLM"	Keep — this is a hard requirement, not a nice-to-have
Full admin dashboard, 12 chart types	Nothing stated — "watching/monitoring" is listed generically, not with this level of detail	Cut to 3-4 charts that map to the actual non-functional targets (latency, cost, error rate); the rest is speculative polish

Direction 2 — every non-functional requirement has something addressing it (the reverse check):

Requirement	What in the design addresses it?	Gap?
p95 latency under 2s	Cache + async tool calls	Covered
Cost under $200/month	*(nothing explicit yet)*	**Gap** — no component in the current design tracks or caps per-request cost. This needs to be added, not just noted.
Never expose raw customer data to the LLM	Redaction layer	Covered
Reliability target *(not yet specified)*	*(depends on live handoff)*	Flag as open until the number is known

Why this matters: the forward check (Direction 1) catches unjustified additions — the failure mode of over-building. The reverse check (Direction 2) catches missing coverage — the failure mode of under-building, which is just as common and easier to miss because nothing "looks wrong" about a design that's simply incomplete. Finding the cost-tracking gap here, in writing, before any code exists, is exactly the value this exercise is meant to deliver.